

BOTBOND · BONDED AGENT ACCESS

처음 보는 AI 에이전트도 가입 없이 필요한 API만 쓸 수 있어야 합니다

BotBond는 공급자가 직접 설치하는 에이전트 접근 게이트웨이입니다. 에이전트가 할 일을 설명하면, 공급자는 그 작업에 필요한 API 권한·호출 한도·비용을 정해 즉시 세션을 엽니다.

예약이나 주문처럼 나중에 책임이 생기는 작업은 환불 가능한 온체인 담보로 관리합니다.

Permissionless agent onboarding for structured APIs · Google Cloud × Solana

API key만으로는 일회성 에이전트를 받기 어렵습니다

공급자

가격·재고·견적·예약 API
MCP와 전문 데이터 공급자

에이전트

HTTP/MCP 요청과 지갑을 제어하
는 구매·조달·여행·운영 에이전트

오늘의 선택지

알 수 없는 자동화를 막거나, 가입·심
사·계약을 거친 뒤 장기 API key를
발급

한 번의 가격 비교나 재고 확인을 하려는 에이전트에게 장기 계정은 과합니다. 반대로 공급자는 아무 에이전트에게나 API를 열 수 없습니다.

BotBond는 이 사이에 있는 **짧고 제한적인 공식 접근 경로**입니다.

제외 대상: 학습 크롤러, 은폐형 악성 봇, 임의 사이트 우회, 운영자가 통합하지 않은 웹페이지

기존 보안과 결제 사이에 빈 자리가 있습니다

방식	잘하는 일	남는 공백
Cloudflare / WAF	비협조적 트래픽 차단	협조적인 미등록 에이전트의 온보딩
API key	장기 고객 인증·청구	일회성 접근의 가입·심사 비용
Rate limit	호출량 통제	자연어 목적과 endpoint 의미
pay-per-use	데이터 사용료	예약 등 사후 의무의 담보
Web Bot Auth	알려진 에이전트의 신원 확인	미등록 에이전트가 즉시 책임을 보일 방법

BotBond는 WAF나 API key를 대체하지 않습니다. 장기 고객은 API key를 쓰고, 비협조적 트래픽은 WAF가 막습니다. BotBond는 그 사이에서 처음 보는 에이전트에게 필요한 만큼만 접근권을 발급합니다.

제품은 네 가지 경로에서 작동합니다

/shop Customer Shop

사람은 평소처럼 상품을 보고 구매합니다. 일반 고객 흐름에는 BotBond가 나타나지 않습니다.

/agent External Agent

처음 보는 에이전트는 거부 응답에서 공식 진입점을 발견하고, 작업·권한·담보 조건을 확인합니다.

/merchant Merchant Ops

운영자는 허용 범위, 세션 상태, 예약, 반환·정산 내역을 봅니다.

/integrate Developer Setup

공급자는 discovery 문서와 예제로 자신의 에이전트 또는 API를 연결합니다.

Overview는 이 네 경로와 작동 원리를 소개하는 시작 화면입니다.

일반 API는 닫아두고 에이전트 경로만 따로 엽니다

일반 자동화 요청

```
Agent
→ Cloudflare / WAF
→ 기존 정책 또는 차단
```

운영자는 기존 보안 정책을 그대로 유지합니다.

운영자가 연 에이전트 경로

```
Agent
→ WAF allowlisted path
→ BotBond Gateway
→ Scoped Origin API
```

작업과 조건에 동의한 에이전트만 이 경로로 들어옵니다.

WAF를 건너뛰면 공급자는 기존 보안 정책과 접근 통제권을 잃습니다. BotBond가 차단을 풀어 주면 악성 자동화도 같은 문을 사용할 수 있습니다. 그래서 BotBond는 WAF 뒤에서, 운영자가 허용한 경로에만 붙습니다. 이 경로에서는 처음 보는 에이전트도 정해진 조건으로 세션을 열 수 있습니다.

일반 웹/API는 계속 닫아두고, `/.well-known/agent-access` 와 범위가 제한된 에이전트 API만 별도 정책으로 엽니다.

데모의 최초 403은 **BShop edge policy 재현**이며 실제 Cloudflare zone 이벤트가 아닙니다.

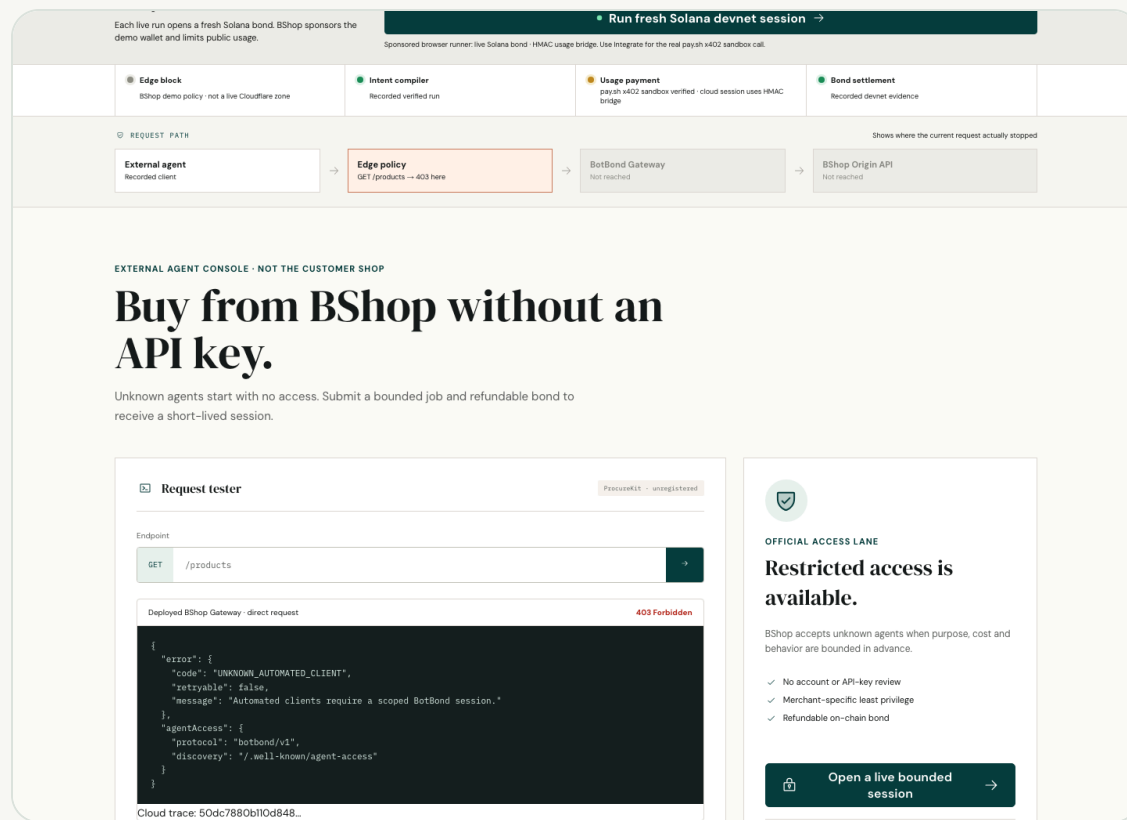
실제 요청도 일반 경로에서 멈춥니다

미등록 agent의 직접 요청

배포된 BShop Gateway는 `GET /products` 를 `403` 으로 돌려보냅니다. Origin API에는 요청이 전달되지 않습니다.

응답에는 우회 방법 대신 공식 discovery 경로 `/.well-known/agent-access` 가 들어 있습니다. 에이전트는 이 경로에서 조건을 확인한 뒤에만 다음 단계로 갑니다.

이 캡처는 live Gateway 요청입니다. BShop의 edge policy를 보여 주며, 실제 Cloudflare zone 로그라고 주장하지 않습니다.



직접 요청 `403` · discovery 경로 반환 · origin 미도달

에이전트는 API 구조 대신 할 일을 설명합니다

에이전트의 작업 요청

1,500 이하 노트북의 가격과 재고를 비교하고 마지막 한대를 예약한 뒤 해제해줘. 판매자 연락처와 리뷰는 필요 없어.

공급자 정책 초안

```
{
  "allowed": [
    "GET /products",
    "GET /products/{id}/inventory",
    "POST /reservations",
    "POST /reservations/{id}/release"
  ],
  "max_calls": 25,
  "reservation_ttl": "60s"
}
```

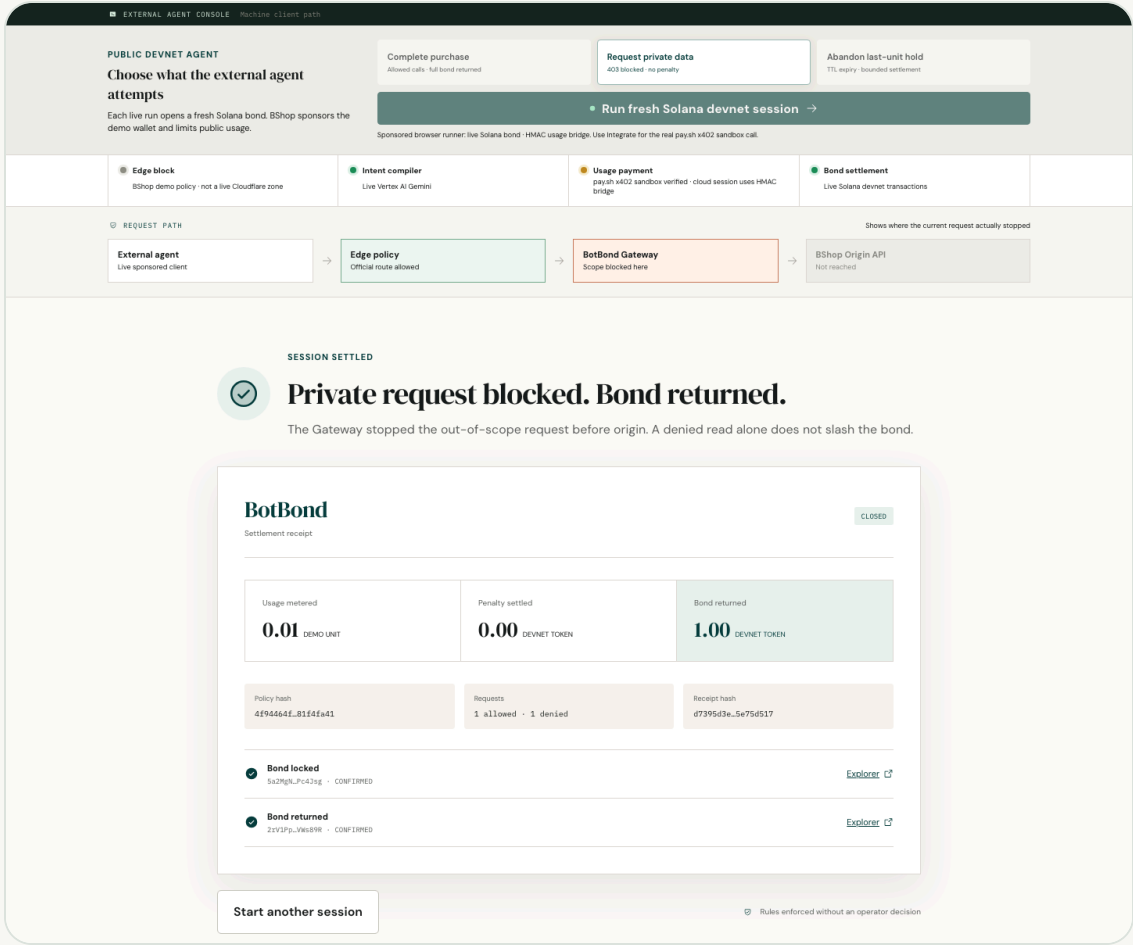
같은 가격·재고 비교라도 상점마다 endpoint와 field 이름이 다릅니다. 에이전트가 API 구조를 직접 추측하면 공급자 문서를 매번 학습해야 하고, 넓은 권한을 요구하기 쉽습니다. BotBond는 에이전트의 작업 설명과 공급자가 제공한 API catalog를 맞춰, 그 작업에 필요한 범위만 담은 정책 초안을 만듭니다.

Gemini는 정책 초안을 만들고, Gateway가 endpoint·field·rate·TTL을 결정론적으로 집행합니다. 세션이 시작된 뒤 AI가 권한을 넓히거나 돈을 정산하지 않습니다.

세션 안에서는 약속한 범위만 실행됩니다

시도	결과	Origin 도달	담보 변화
session 없이 재고 조회	Edge policy 403	아니오	없음
허용된 가격·재고 조회	Gateway 200	예, 허용 field만	없음
seller contacts 요청	Scope 403	아니오	없음
예약 후 정상 해제	Session close	예	전액 반환
예약 후 TTL 방치	Objective expiry	예약만 도달	0.25 제한 정산

차단은 몰수가 아닙니다. 실제 비용을 만든 bonded action의 객관적 위반만 사전에 서명한 상한 안에서 정산합니다.



범위 밖 private request는 Gateway에서 차단되고, bond는 전액 반환됩니다.

요청부터 정산까지 역할을 나눴습니다

```
Sponsored Browser Agent — HMAC demo bridge ┐
Own-wallet Agent CLI — pay.sh x402 sandbox —┐ BotBond Gateway · Cloud Run
                                              └ Vertex AI Gemini → policy
                                                └ Firestore → session + evidence
                                                  └ deterministic guard → scope / rate / TTL
                                                    └ BShop Origin API → price / inventory
                                                      └ Solana devnet → bond / refund / settlement
```

브라우저는 Firestore·서명 키에 직접 접근하지 않습니다. **pay.sh 결제는 own-wallet CLI만 수행**하며, browser는 HMAC demo bridge를 명시적으로 표시합니다. Solana settlement는 confirmed transaction만 Explorer에 연결합니다.

지금 동작하는 범위와 데모 경계

구성	상태	검증 증거
Vertex AI Gemini	LIVE	실행마다 새 policy와 hash
Cloud Run + Firestore	LIVE	public API, session state, events
Solana bond program	LIVE DEVNET	open + refund/settlement tx
pay.sh per-call rail	HOSTED SANDBOX	실제 402 → pay → 200
Browser session activation	DEMO BRIDGE	HMAC credential, pay.sh verifier 아님
Cloudflare zone/WAF	NOT INTEGRATED	BShop Gateway가 403 정책만 재현

온체인 자산은 **devnet test mint**이며 실제 USDC가 아닙니다.

브라우저와 외부 에이전트의 결제 경로를 구분합니다

The screenshot displays the BShop public integration surface, which guides users through the payment process. It is divided into several sections:

- Normal web traffic**: Shows the agent path from Cloudflare WAF to the existing site policy, noting that unknown automation remains blocked.
- Merchant-approved agent lane**: Shows the agent path from WAF through an allowlisted path to a BotBond-scoped API, noting that no Cloudflare bypass occurs.
- LIVE CONNECTION CHECK**: A section titled "Test BShop's public integration surface." with a "Run connection check" button. It contains three panels:
 - Agent discovery (200)**: GET /.well-known/agent-access. Program EGS&KPv6, published by discovery. Cloud trace 79a78ac254b7502.
 - Direct product request (403)**: GET /products. UNKNOWN_AUTOMATED_CLIENT. Cloud trace 4334cc791a57096.
 - pay.sh payment gate (402)**: GET /v1/access/browser-check/products. x402 payment challenge returned. Cloud trace 79a78ac254b7502.
- OWN-WALLET EXECUTION**: A section titled "Run the paid sandbox rail and devnet bond yourself." with a "Copy external-agent command" button. It contains three steps:
 - 01 Discover**: GET /.well-known/agent-access. Read the catalog, session endpoints, Solana program and devnet onboarding mode.
 - 02 Declare intent**: POST /v1/intents. Gemini maps the request to BShop's own endpoint and field catalog.
 - 03 Open the bond**: The example creates a devnet test mint, opens the real bond, activates a short session and prints Explorer links.

Browser sponsored run

Gemini, Gateway, Firestore와 Solana devnet bond를 실제로 실행합니다. 사용량 credential은 HMAC demo bridge이며 pay.sh verifier가 아닙니다.

Own-wallet agent CLI

자기 Solana 지갑으로 pay.sh hosted sandbox의 402 → payment → scoped 200 을 수행합니다. 성공하면 상품 응답과 bond open·정산 Explorer 링크가 출력됩니다.

한 화면에서 서로 다른 rail을 하나의 결제로 보이게 하지 않습니다.

공개 connection check: discovery 200 · 직접 요청 403 · pay.sh gate 402

개발자는 discovery에서 세션 발급과 호출을 시작합니다

1. 공개 정보 조회

```
GET /.well-known/agent-access
```

intent-session endpoint, payment rail, Solana program ID와 지원 범위를 받습니다.

2. 작업과 지갑 제출

```
POST /v1/intents  
{ task, wallet, budget }
```

Gemini가 공급자 catalog 안에서 policy와 policy hash를 만듭니다. 에이전트는 담보를 열기 전에 조건을 검토합니다.

3. 세션으로 호출

```
x-botbond-session-token: <token>  
GET /v1/access/:sessionId/products
```

정책 hash로 devnet bond를 연 뒤 세션을 발급받습니다. 허용된 endpoint만 호출할 수 있습니다.

가장 쉬운 실행 방법은 `/integrate`의 `own-wallet` command입니다. 이 명령은 discovery부터 `pay.sh` sandbox 호출, `bond open·close`, Explorer URL 출력까지 실행합니다.

정산 결과는 Solana Explorer에서 누구나 확인합니다

SOLANA EXPLORER (BETA) Search for tokens, validators, programs, and accounts

Feature Gates Inspector Devnet

DETAILS Transaction

Summary View Receipt Inspect Refresh Download

Status	Success
Confirmation	Finalized (MAX Confirmations)
Signature	4kkFpZM3DNzDEYqTuA9eXd7fsYv6klvZRRV7posqddUUViFxFvTJdKPRTDskdcFKNJmFzQ8Tz3R5Zoj3m961
Fee payer	GMrS1AR2MhVW9cDmQJ4RApQz6iTS17srfb8bwucJ0a6
Slot	480,916,681
Recent Blockhash	DXfgDKT4FQzymEo1LRp94cBUTUgVT9MYQRboVt14qQE
Fee	0.000005
Transaction cost	45,521
CU's Consumed / Limit	43,471 / 200,000
Transaction Version	LEGACY
Timestamp	Aug 3, 2026 at 21:37:00 Korean Standard Time

Summary Accounts Tokens Programs & Logs

Accounts & SOL balance Collapse

#	ADDRESS	CHANGE (SOL)	POST BALANCE (SOL)
1	GMrS1AR2MhVW9cDmQJ4RApQz6iTS17srfb8bwucJ0a6 Fee Payer (Signer) Writable	-0.00461948	0.18.23972688
2	37ta2VP5Q3wzCxhvXdtBBtNj058RTnHFWSbXyBb85p6 Writable	+0	0.00283928
3	3mfpcFQVY7HSFEph5azZk1fCwK1SG5BN4w5kVmbk8 Writable	+0.00283928	0.00283928
4	4gdCz2TQe1xnF5saQN7yNm5aq2CUdFE1dEHkmXsZCUDG	+0.0025752	0.0025752

확인 순서

1. BotBond 영수증의 **Bond locked** 또는 **Bond returned** Explorer 링크를 엽니다.
2. Explorer 상단에서 **Success / Finalized** 상태와 transaction signature를 확인합니다.
3. Accounts & SOL Balance에서 계정 변화를, Programs & Logs에서 BotBond program 실행 기록을 봅니다.

이 화면이 보여 주는 것

담보 잠금·반환·제한 정산은 운영자 대시보드의 숫자만이 아니라 공개 devnet transaction으로 검증됩니다.

이 캡처는 devnet test mint 기준입니다. pay.sh 반복 세션 검증이나 실제 USDC 거래를 뜻하지 않습니다.

3분 영상에서 제품 흐름을 보여줍니다

1. 사람의 쇼핑

`/shop` 에서 일반 고객은 기존처럼 상품을 보고 구매합니다. BotBond는 이 경로를 바꾸지 않습니다.

2. 에이전트의 첫 요청

`/agent` 에서 미등록 에이전트가 일반 API를 요청해 `403` 을 받고, discovery 문서에서 공식 에이전트 경로를 찾습니다.

3. 제한된 세션

에이전트가 작업을 설명하면 policy·담보 조건이 제시됩니다. browser flow는 새 Solana devnet bond와 scoped `200` , private `403` 을 보여줍니다.

4. 정산과 외부 에이전트

정상 종료는 반환, 예약 방치는 제한 정산으로 끝납니다. 별도 terminal에서는 own-wallet 에이전트가 pay.sh sandbox의 `402 → payment → 200` 을 실행합니다.

영상의 browser flow와 own-wallet pay.sh flow는 서로 다른 실행 경로로 표시합니다. 하나의 세션처럼 편집하지 않습니다.

심사위원이 직접 확인할 수 있습니다

라이브 제품

`botbond-bshop.vercel.app`

Shop · Agent · Merchant · Integrate

내 지갑으로 실행

```
npm run example:external-agent -- \
  --gateway https://... \
  --wallet ~/.config/solana/id.json
```

제출물: 발표 PDF · 재현 가능한 GitHub/README · 3분 실제 온체인 영상 · 심사 기간 접근 가능한 endpoint

Program: `EG9r...KaRKR` · asset: devnet test mint, not USDC

처음 보는 에이전트에게 필요한 것은 즉시 신뢰가 아니라, 범위가 분명하고 정산 규칙이 정해진 접근권입니다.